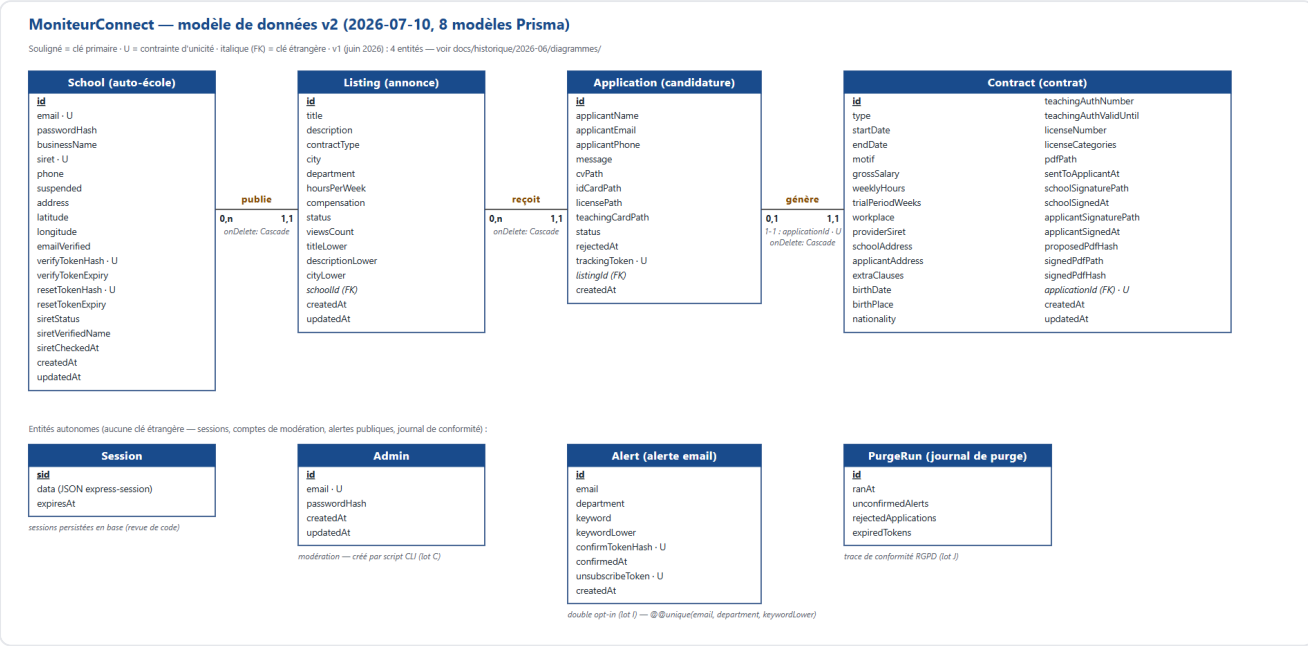


Base de données — modèle v2

Date : 2026-07-10 (schéma éclaté en multi-fichiers le 2026-07-12). Source de vérité : `prisma/schema/` — un modèle par fichier commenté, `datasource` et `generator` dans `main.prisma` — et les 13 migrations de `prisma/schema/migrations/`. Le MCD/MLD initial (4 entités, juin 2026) est conservé sous `../historique/2026-06/diagrammes/`.



Source vectorielle : `diagrammes/bdd-v2.svg`.

Lecture du diagramme

La rangée du haut est le cœur relationnel : une **School** publie 0-n **Listing**, un Listing reçoit 0-n **Application**, une Application génère au plus un **Contract** (relation 1-1 garantie par `applicationId @unique`). Toutes ces relations sont en suppression **en cascade** : supprimer une annonce emporte ses candidatures et leurs contrats — le code supprime d'abord les fichiers privés associés (voir `test/smoke.cjs`, assertions A1).

La rangée du bas regroupe les entités **autonomes** (aucune clé étrangère) :

- **Session** : sessions express persistées en base (survivent aux redémarrages, identiques en SQLite et PostgreSQL) ;
- **Admin** : comptes de modération, créés par script CLI uniquement ;
- **Alert** : alertes email publiques en double opt-in, dédoublées par la contrainte composée `@@unique(email, department, keywordLower)` ;
- **PurgeRun** : journal des purges RGPD, trace de conformité affichée sur le dashboard admin.

Évolution : de 4 à 8 entités

Entité / colonnes	Origine	Rôle
School , Listing , Application , Contract	MCD v1 (juin)	Cœur métier, inchangé dans sa structure
School.address/latitude/longitude	MVP carte	Localisation géocodée (Nominatim)
Listing.*Lower	Lot A	Recherche insensible à la casse, portable SQLite/PostgreSQL
Application.trackingToken	Lot B	Suivi candidat sans compte, jeton opaque unique
School.suspended + Admin	Lot C	Modération et suspension
Session	Revue de code	Sessions en base au lieu de la mémoire
School.siretStatus/siretVerifiedName/siretCheckedAt	Lot F	Vérification Sirene, jamais bloquante
7 colonnes de signature sur Contract	Lot G	Signatures, horodatages, empreintes SHA-256
Listing.viewsCount	Lot H	Compteur de vues, incrément fire-and-forget
Alert	Lot I	Alertes email double opt-in
Application.rejectedAt + PurgeRun	Lot J	Point de départ et journal de la purge RGPD

Trois contraintes d'intégrité à citer au jury

1. **School.siret @unique** : deux écoles ne peuvent pas partager un SIRET ; la collision en course retombe sur l'erreur Prisma `P2002`, transformée en message utilisateur (testé dans `test/correctifs.cjs`).
2. **Contract.applicationId @unique** : le 1-1 est garanti par la base, pas seulement par le code — impossible de générer deux contrats pour une même candidature (testé dans `test/lot-g.cjs`).
3. **Cascades onDelete** : School → Listing → Application → Contract ; l'intégrité référentielle ne laisse jamais d'orphelins, et le code nettoie les fichiers privés avant la suppression (testé dans `test/smoke.cjs`).

SQLite en développement, PostgreSQL en production

Le code applicatif utilisant Prisma est conçu pour rester portable, mais les **migrations ne le sont pas** : l'historique SQL actuel cible SQLite et ne peut pas être rejoué tel quel sur PostgreSQL. Le passage en production demande donc un provider et une `DATABASE_URL` PostgreSQL, mais aussi une chaîne de migrations PostgreSQL dédiée et testée.

Les choix qui limitent les écarts de code entre les deux moteurs sont :

- recherche insensible à la casse par colonnes `*Lower` maintenues à l'écriture (le `LOWER()` SQL se comporte différemment selon les moteurs) ;
- bucketing des séries hebdomadaires en JavaScript plutôt qu'en SQL dépendant du dialecte (lot H) ;
- sessions et jetons stockés en colonnes portables (chaînes, dates).

Création et migration en développement

```
copy .env.example .env
npx prisma migrate deploy
npx prisma generate
npm run seed:demo          # démonstration uniquement, jamais en production
```

`prisma/schema/migrations/` reste l'historique SQLite de développement. Chaque modification commence dans le fichier concerné de `prisma/schema/`, reçoit une migration versionnée, puis passe `npx prisma validate` et `npm test`.

Préparation de PostgreSQL

La première mise en production doit être répétée sur une base PostgreSQL vide :

1. créer une configuration Prisma dédiée dont le datasource utilise `provider = "postgresql"` avec l'adaptateur `@prisma/adaptateur-pg` (le projet est en Prisma 7 depuis le 2026-07-12 : la connexion passe par un adaptateur de driver — `better-sqlite3` en développement), sans mélanger ses migrations avec celles de `prisma/schema/migrations/` ;
2. générer une migration de référence PostgreSQL à partir du schéma actuel avec `prisma migrate diff --from-empty --to-schema-datamodel ... --script` ;
3. relire le SQL produit (types, index, cascades et contraintes uniques), puis le versionner dans l'historique PostgreSQL dédié ;
4. appliquer cet historique sur une base de préproduction avec `npx prisma migrate deploy --schema <schema-postgresql>` ;
5. générer le client, exécuter les 15 suites de tests et contrôler les parcours inscription → annonce → candidature → contrat signé ;
6. seulement après cette répétition, appliquer la même version en production.

Il n'existe encore aucun déploiement PostgreSQL réel dans le dépôt : cette partie reste une trajectoire documentée, pas une réalisation à survendre au jury.

Sauvegarde et restauration

Toujours arrêter les écritures applicatives (mode maintenance ou serveur arrêté), dater le fichier de sauvegarde et restaurer d'abord vers une **base séparée**. Une sauvegarde non restaurée au moins une fois n'est pas une preuve de reprise.

SQLite local

La commande `.backup` produit une copie cohérente même avec le journal SQLite :

```
New-Item -ItemType Directory -Force backups
sqlite3 prisma/dev.db ".backup 'backups/moniteur-connect-20260710.sqlite'"

# Restauration de contrôle dans un autre fichier
sqlite3 prisma/restauration-controle.db ".restore 'backups/moniteur-connect-20260710.sqlite'"
sqlite3 prisma/restauration-controle.db "PRAGMA integrity_check;"
```

Le dernier résultat doit être `ok` . Pointer temporairement `DATABASE_URL` vers `restauration-controle.db` , démarrer l'application et vérifier les volumes et un parcours métier avant de considérer la sauvegarde comme exploitable. Ne jamais écraser `dev.db` pendant un exercice.

PostgreSQL cible

Les outils natifs conservent le schéma et les données. Les identifiants doivent venir du gestionnaire de secrets ou de `.pgpass` , jamais d'un fichier versionné :

```
pg_dump --format=custom --no-owner --file=backups/moniteur-connect-20260710.dump
$env:DATABASE_URL

# RESTORE_DATABASE_URL pointe vers une base vide de contrôle
pg_restore --clean --if-exists --no-owner --dbname=$env:RESTORE_DATABASE_URL backups/moniteur-
connect-20260710.dump
```

Après restauration : exécuter une requête de comptage sur les entités clés, `npx prisma migrate status --schema <schema-postgresql>` , puis les tests et un smoke test applicatif. La durée, le résultat et la personne ayant réalisé l'exercice doivent être consignés ; ce test de restauration reste à effectuer lorsqu'un environnement PostgreSQL sera disponible.